nolibc: a userspace libc in the kernel tree

Thomas Weißschuh (Linutronix) Willy Tarreau (HAProxy)

Kernel Recipes 2025, 2025-09-24

2025-09-24



About me

- Kernel hobbyist since 2018
- Occasional work-related patches
- Maintainer of
  - nolibc
  - multiple ChromeOS Embedded Controller drivers
  - multiple hwmon drivers
- Hardening patchsets
- Hardware enablement
- Bug fixes
- Working for Linutronix on the kernel since 2024

Overview

- Small, header-only C standard library
- Initially designed for early userland init code
- Now maintained as part of the kernel tree
- Licensed as LGPL-2.1 or MIT
- Easily extensible, both with new features and architectures

From: "Paul E. McKenney" <paulmck@linux.vnet.ibm.com>
Subject: Kernel-only deployments?
Date: Thu, 23 Aug 2018 10:43:59 -0700
Cc: linux-kernel@vger.kernel.org

Does anyone do kernel-only deployments, for example,
setting up an embedded device having a Linux kernel
and absolutely no userspace whatsoever?
(...)
The mkinitramfs approach results in about 40MB of initrd,
and dracut about 10MB. Most of this is completely useless
for rcutorture, which isn't interested in mounting filesystems,
opening devices, and almost all of the other interesting things
that mkinitramfs and dracut enable.
(...)
I started by throwing out everything not absolutely needed by
the dash and sleep binaries, which got me down to about 2.5MB,
1.8MB of which was libc.

History

- Entered the kernel tree as part of rcutorture in 5.0, maintained by Willy with process support from Shuah and Paul
- Moved to `tools/include/nolibc/` in 5.1
- Prefer kernel UAPI definitions over custom ones
- Thomas became a co-maintainer in 6.5
- Shuah and Paul handed over their process tasks in 6.16
- A total of 22 contributors so far
- Today around ~5000 lines of code

Goals

- Small binary size through
  - simple source code
  - compiler optimizations
- Easy to understand, both the source and object code
- Stay close to a “real” libc
- Always statically linked
- No implicit allocations
- Multi-architecture support from a single codebase
- Can be vendored into other projects

Requirements

- C compiler
  - Tested with GCC and clang
  - A kernel toolchain without libc is sufficient [1]
- Kernel UAPI headers for the correct architecture
  - Built directly from the kernel sources
  - From the systems default headers in `/usr/include/`
- A simple enough application

[1] Arnd Bergmann's kernel.org crosstools work great

Usage

```
$ cat hello.c
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");

    return 0;
}
```

```
$ cc -nostdinc -nostdlib -static \
  -Iusr/include/ -Itools/include/nolibc/ \
  hello.c -o hello
$ ./hello
Hello World!
```

System calls

- Lowlevel I/O
- File management
- Process control and management
- Clocks and timers
- `ioctl()`
- `syscall()`

Standard library

- Integer types
- `printf()`/`scanf()` and friends
- POSIX file streams
- Math functions
- String functions
- Simple malloc()
- getopt()
- `__attribute__((constructor))`/getauxval()

Architecture support

- Each architectures specific code is contained in a single file
- System call wrappers: `my_syscall0()` to `my_syscall6()`
- Assembly `_start()` entrypoint
- ~200 lines per architecture
- Fair amount of supported architectures
  - x86 (32bit, 64bit, x32)
  - ARM (32bit, 64bit, thumb)
  - RISC-V (32bit, 64bit)
  - PowerPC (32bit, 64bit, little-endian, big-endian)
  - MIPS (o32, n32, n64, little-endian, big-endian)
  - s390 (32bit, 64bit)
  - loongarch (64bit), SPARC (32bit, 64bit), sh4, m68k

Architecture support (arm64 system call wrapper)

```
#define my_syscall2(num, arg1, arg2)
({
    register long _num  __asm__ ("x8") = (num);
    register long _arg1 __asm__ ("x0") = (long)(arg1);
    register long _arg2 __asm__ ("x1") = (long)(arg2);

    __asm__ volatile (
        "svc #0\n"
        : "=r"(_arg1)
        : "r"(_arg1), "r"(_arg2),
          "r"(_num)
        : "memory", "cc"
    );
    _arg1;
})
```

Architecture support (arm64 entrypoint)

```
void
__attribute__((weak, noreturn))
__nolibc_entrypoint __no_stack_protector
_start(void)
{
    __asm__ volatile (
      /* save stack pointer to x0, as arg1 of _start_c */
      "mov x0, sp\n"
      /* transfer to c runtime                          */
      "bl  _start_c\n"
    );
    __nolibc_entrypoint_epilogue();
}
```

System call wrappers

```c
#include "../sys.h"
#include <linux/timerfd.h>

static __attribute__((unused))
int sys_timerfd_create(int clockid, int flags)
{
    return my_syscall2(__NR_timerfd_create,
                       clockid, flags);
}

static __attribute__((unused))
int timerfd_create(int clockid, int flags)
{
    return __sysret(sys_timerfd_create(clockid, flags));
}
```

Current in-tree users

- rcutorture, the original user
- riscv selftests
- arm64 selftests
- vDSO selftests
- Kexec HandOver (KHO) tests
- Works with `kselftest.h`
- Works with `kselftest_harness.h`, might require libgcc

Limitations

- No pthreads, thread-locals
- No longjmp()
- No signals (yet)
- No networking
- Not y2038-safe on all architectures
- stdlib and systemcall wrappers are not complete
- The code is optimized for size over performance
- Only usable for complete executables
- Not all Linux' architectures are supported (yet)

Size comparison

```
$ gcc -Os hello.c -o hello-glibc
$ gcc -Os -static hello.c -o hello-glibc-static
$ musl-gcc -Os hello.c -o hello-musl
$ gcc -Os hello.c -o hello-nolibc \
    -nostdinc -nostdlib -static \
    -Itools/include/nolibc/ -Iusr/include/

$ size hello-*
   text    data     bss     dec     hex filename
   1297     584       8    1889     761 hello-glibc
 695713   23720   22576  742009   b5279 hello-glibc-static
   4179     336    1688    6203    183b hello-musl
   2310     360      24    2694     a86 hello-nolibc
```

Testsuite

- Custom test framework and runner
- Different architectures through QEMU
- Tests both GCC and clang
- The testsuite itself is tested against the system libc
- Should migrate to `kselftest.h` for TAP output

Outlook

- Additional architectures
- Patches for signal support are under review
- Patches for User Mode Linux are under review
- Multi-architecture sysroot support
- Integration with KUnit and kselftests for UAPI testing
- libgcc implementation

Excursion: Testing the generic vDSO library

- Started working on the vDSO
- Migrating architecture-specific code to a generic library
- Needs to be tested
- No built-in support for cross-testing in kselftests
- Experimentation for a usable test setup
  - nolibc-test provides cross-architecture scaffolding
  - Manually transplant vDSO selftests
  - Ugly, not upstreamable

Kunit UAPI tests

- Integration of kselftests into KUnit
- Build kselftests as `userprogs`
- nolibc for kernel toolchain compatibility
- Embed the test binaries in the kernel image or module
- Use KUnit to drive the test execution
- Embed the TAP output from kselftests into KUnit's KTAP
- Unified execution and reporting

Example UAPI test

```c
#include "../../tools/testing/selftests/kselftest.h"

int main(void)
{
    ksft_print_header();
    ksft_set_plan(4);
    ksft_test_result_pass("userspace test 1\n");
    ksft_test_result_pass("userspace test 2\n");
    ksft_test_result_skip("userspace test 3: some reason\n");
    ksft_test_result_pass("userspace test 4\n");
    ksft_finished();
}
```

```
$ ./tools/testing/kunit/kunit.py run --arch x86_64 --kunitconfig lib/kunit example
[18:09:17] Configuring KUnit Kernel ...
[18:09:23] Building KUnit Kernel ...
$ make all compile_commands.json scripts_gdb ARCH=x86_64 O=.kunit --jobs=16
[18:10:14] Starting KUnit Kernel (1/1)...
Running tests with:
$ qemu-system-x86_64 -kernel .kunit/arch/x86/boot/bzImage -append 'kunit.filter_glob=example
[18:10:14] ================== example (10 subtests) ==================
[18:10:14] [PASSED] example_simple_test
(...)
[18:10:14] [PASSED] example_slow_test
[18:10:14] ====================== (4 subtests) ======================
[18:10:14] [PASSED] userspace test 1
[18:10:14] [PASSED] userspace test 2
[18:10:14] [SKIPPED] userspace test 3: some reason
[18:10:14] [PASSED] userspace test 4
[18:10:14] =============== [PASSED] example_uapi_test ===============
[18:10:14] ==================== [PASSED] example ====================
[18:10:14] ============================================================
[18:10:14] Testing complete. Ran 16 tests: passed: 11, skipped: 5
[18:10:14] Elapsed time: 57.500s total, 5.571s configuring, 51.008s building, 0.520s running
```

```
$ ./tools/testing/kunit/kunit.py run (...) --raw_output=kunit example
KTAP version 1
1..1
    # example: initializing suite
    KTAP version 1
    # Subtest: example
    # module: kunit_example_test
    1..10
(...)
    ok 9 example_slow_test
    # example_uapi_test: initializing
        TAP version 13
        1..4
        ok 1 userspace test 1
        ok 2 userspace test 2
        ok 3 # SKIP userspace test 3: some reason
        ok 4 userspace test 4
        # Totals: pass:3 fail:0 xfail:0 xpass:0 skip:1 error:0
    # example_uapi_test: cleaning up
    ok 10 example_uapi_test
    # example: exiting suite
# example: pass:8 fail:0 skip:2 total:10
# Totals: pass:9 fail:0 skip:4 total:13
ok 1 example
```

Links

- Original mail from Paul: 20180823174359.GA13033@linux.vnet.ibm.com
- nolibc in the kernel tree: tools/include/nolibc/
- nolibc on LWN: https://lwn.net/Articles/920158/
- KUnit UAPI tests on LWN: https://lwn.net/Articles/1029077/
- KUnit UAPI tests on LKML: 20250717-kunit-kselftests-v5-0-442b711cde2e@linutronix.de

Thank you!

- Willy Tarreau <w@1wt.eu>
- Thomas Weißschuh <linux@weissschuh.net>